Polymorphism

method overriding

```
class Animal:
    def speak(self):
        return 'Sound of ...'

class Dog(Animal):
    def speak(self):
        return 'Woof'

class Cat(Animal):
    def speak(self):
        return 'Meow'
```

3 different declarations of same fn().

eg →

```
dog = Dog()
cat = Cat()
```

```
dog.speak() ——→ Woof
cat.speak() ——→ Meow
```

Note: With the help of overriding, we can have respective declarations of different classes

eg of polymorphism

```
def animal_speaks(animal):
    print(animal.speak())

print(animal_speaks(dog))
```

argument is object of any class

speak() defined above.

object made above

public function

→ Woof

Abstract Classes

→ from abc import ABC, abstractmethod

```
class Vehicle (ABC):
    @ abstractmethod
    def start_eng (self):
        pass
```

} abstract class. (Parent)

→ cannot create instances of this class.
→ can only be inherited
→ They ~~gua~~ guarantee that subclasses implement certain methods.

```
class ~~car~~ Car (Vehicle):
    def start_eng (self):
        return "Started".
```

← ~~with~~ must name abstract methods ~~define~~ declared in parent class

Encapsulation

(To restrict direct access to some of the object's components)

```
class Person:
    def __init__ (self, name, age):
        self.__name = name
        self.__age = age
        self.gender = gender
person = Person ('Viv', 25)
```

} private variables
↳ declare by using self.__var = var.

→ dir (person)
→ will show the publicly accessible attributes, but not the private variables.

→ __name
__age } cannot be accessed outside the class definition.

Protected variable	Private variable
⇒ self._name = name	self.__name = name
⇒ can be accessed only from a derived class	⇒ cannot be accessed outside the class definition
⇒ can be accessed by object._name of the derived class.	⇒ object.__name will throw an error.

Getter & Setter methods

eg

```
class Person:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age

    # getter eg:
    def get_name(self):
        return self.__name

    def get_age(self):
        return self.__age

    def set_name(self, name):
        self.__name = name

    def set_age(self, age):
        self.__age = age
```

getter methods

setter methods

private access modifier/variable

double __

double __ access